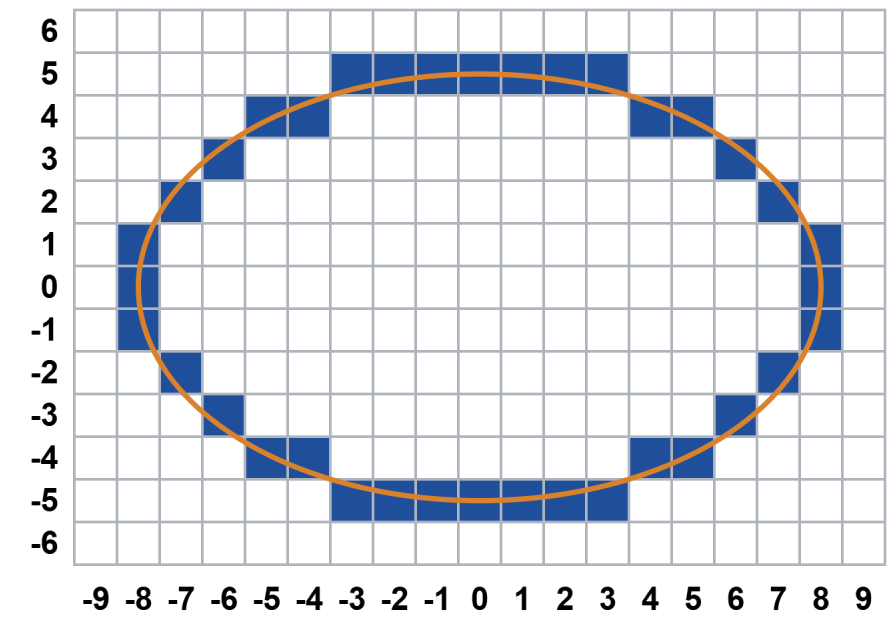
0613-403 · Week 3 · Lecture 6

Midpoint Ellipse, Boundary Fill and Flood Fill

Ekramul Haque Amin

Lecturer, AI and Data Science

Dhaka International University



Review problems: solve before we start

Q1 · DDA

Find the pixels of the line from (1, 2) to (7, 6).

Q2 · Bresenham

Find the pixels of the line from (3, 2) to (12, 7).

Q3 · Midpoint line

Find the pixels of the line from (0, 0) to (8, 3).

Q4 · Midpoint circle

Find the first-octant pixels of a circle with r = 9, centre (0, 0).

Show every step in a table, the way you will in the exam. Time: 12 minutes.

Q1: DDA, (1, 2) to (7, 6)

Step along the longer side; add xInc and yInc each step and round.

Step 1: dx = ____, dy = ____

Step 2: steps = max(|dx|, |dy|) = ____

Step 3: xInc = dx ÷ steps = ____

Step 4: yInc = dy ÷ steps = ____

Step 5: start at (1, 2), add the increments each step

Round with floor(v + 0.5).

k	x	y	Pixel (round x, round y)

Q1 solution: DDA, (1, 2) to (7, 6)

Step along the longer side; add xInc and yInc each step and round.

Step 1: dx = 7 − 1 = 6, dy = 6 − 2 = 4

Step 2: steps = max(|6|, |4|) = 6

Step 3: xInc = 6 ÷ 6 = 1

Step 4: yInc = 4 ÷ 6 = 0.67

Step 5: start at (1, 2), add the increments 6 times

Round with floor(v + 0.5).

k	x	y	Pixel (round x, round y)
0	1	2	(1, 2)
1	2	2.67	(2, 3)
2	3	3.33	(3, 3)
3	4	4	(4, 4)
4	5	4.67	(5, 5)
5	6	5.33	(6, 5)
6	7	6	(7, 6)

Q2: Bresenham, (3, 2) to (12, 7)

Integers only: one test and one addition per pixel.

Step 1: dx = ____, dy = ____

Step 2: d_1 = 2dy − dx = ____

Step 3: inc1 = 2dy = ____ (d < 0, choose S)

Step 4: inc2 = 2(dy − dx) = ____ (d ≥ 0, choose T)

Step 5: plot (x, y), test d, update d, x + 1

Check: dx + 1 pixels, last is (12, 7).

x	y	d	Decision	New d

Q2 solution: Bresenham, (3, 2) to (12, 7)

Integers only: one test and one addition per pixel.

Step 1: dx = 12 − 3 = 9, dy = 7 − 2 = 5

Step 2: d_1 = 2dy − dx = 10 − 9 = 1

Step 3: inc1 = 2dy = 10 (d < 0, choose S)

Step 4: inc2 = 2(dy − dx) = −8 (d ≥ 0, choose T)

Step 5: plot (x, y), test d, update d, x + 1

Check: dx + 1 = 10 pixels, last is (12, 7).

x	y	d	Decision	New d
3	2	1	d ≥ 0 → T, y + 1	-7
4	3	-7	d < 0 → S, y stays	3
5	3	3	d ≥ 0 → T, y + 1	-5
6	4	-5	d < 0 → S, y stays	5
7	4	5	d ≥ 0 → T, y + 1	-3
8	5	-3	d < 0 → S, y stays	7
9	5	7	d ≥ 0 → T, y + 1	-1
10	6	-1	d < 0 → S, y stays	9
11	6	9	d ≥ 0 → T, y + 1	1
12	7	—	Stop: reached (12, 7)	—

Q3: midpoint line, (0, 0) to (8, 3)

Test the midpoint M; d is F(M) scaled by 2.

Step 1: dx = ____, dy = ____

Step 2: d_1 = 2dy − dx = ____

Step 3: ΔE = 2dy = ____ (d ≤ 0, choose E)

Step 4: ΔNE = 2(dy − dx) = ____ (d > 0, choose NE)

Step 5: plot, test d, update d, x + 1

Tie rule: at d = 0 choose E.

x	y	d	Decision	New d

Q3 solution: midpoint line, (0, 0) to (8, 3)

Test the midpoint M; d is F(M) scaled by 2.

Step 1: dx = 8, dy = 3

Step 2: $d_1 = 2dy - dx = 6 - 8 = -2$

Step 3: ΔE = 2dy = 6 (d ≤ 0, choose E)

Step 4: ΔNE = 2(dy − dx) = −10 (d > 0, choose NE)

Step 5: plot, test d, update d, x + 1

Note: at d = 0 midpoint picks E; Bresenham would pick T.

x	y	d	Decision	New d
0	0	-2	d ≤ 0 → E, y stays	4
1	0	4	d > 0 → NE, y + 1	-6
2	1	-6	d ≤ 0 → E, y stays	0
3	1	0	d ≤ 0 → E, y stays	6
4	1	6	d > 0 → NE, y + 1	-4
5	2	-4	d ≤ 0 → E, y stays	2
6	2	2	d > 0 → NE, y + 1	-8
7	3	-8	d ≤ 0 → E, y stays	-2
8	3	—	Stop: reached (8, 3)	—

Q4: midpoint circle, r = 9

Compute one octant (x from 0 up to y), then use 8-way symmetry.

Step 1: start at (0, r) = ____

Step 2: $p_0 = 1 - r$ = ____

Step 3: $p < 0 \rightarrow$ E: $p + 2x + 3$

Step 4: $p \geq 0 \rightarrow$ SE: $p + 2(x - y) + 5$, $y - 1$

Step 5: x + 1 each step; stop when x > y

(x, y are the values before the step.)

x	y	p	Decision	New p

Q4 solution: midpoint circle, r = 9

Compute one octant (x from 0 up to y), then use 8-way symmetry.

Step 1: start at (0, r) = (0, 9)

Step 2: $p_0 = 1 - r = 1 - 9 = -8$

Step 3: p < 0 → E: p + 2x + 3

Step 4: p ≥ 0 → SE: p + 2(x − y) + 5, y − 1

Step 5: x + 1 each step; stop when x > y

(x, y are the values before the step.) Next would be (7, 6): x > y, so stop.

x	y	p	Decision	New p
0	9	-8	p < 0 → E, y stays	-5
1	9	-5	p < 0 → E, y stays	0
2	9	0	p ≥ 0 → SE, y − 1	-9
3	8	-9	p < 0 → E, y stays	0
4	8	0	p ≥ 0 → SE, y − 1	-3
5	7	-3	p < 0 → E, y stays	10
6	7	10	p ≥ 0 → SE, y − 1	13

Warm-up: midpoint circle in 3 minutes

Q1

Circle with r = 5. What is p_0?

Q2

Write the 8 symmetric points of (2, 5).

Q3

Why do we stop when x > y?

Today: the same midpoint idea for an ellipse, then how to fill a shape with colour.

Today's plan

Part 1

Midpoint ellipse

- 4-way symmetry
- Two regions
- Proof of every formula
- Worked example and exercise
- C++ code

Part 2

Filling a shape

- 4- and 8-connected pixels
- Boundary fill
- Flood fill
- Problems and comparison

Part 1

The midpoint ellipse algorithm

A circle stretched in one direction

The ellipse equation

Centre at the origin

$$x^2/a^2 + y^2/b^2 = 1$$

a = half-width (along x), b = half-height (along y).

a = b gives a circle.

Without fractions

Multiply both sides by $a^2 \cdot b^2$:

$$b^2x^2 + a^2y^2 = a^2b^2$$

Centre (xc, yc): draw at (0, 0), then add xc and yc to every pixel.

The ellipse function

$$F(x, y) = b^2x^2 + a^2y^2 - a^2b^2$$

$F < 0$

Point is inside the ellipse

$F = 0$

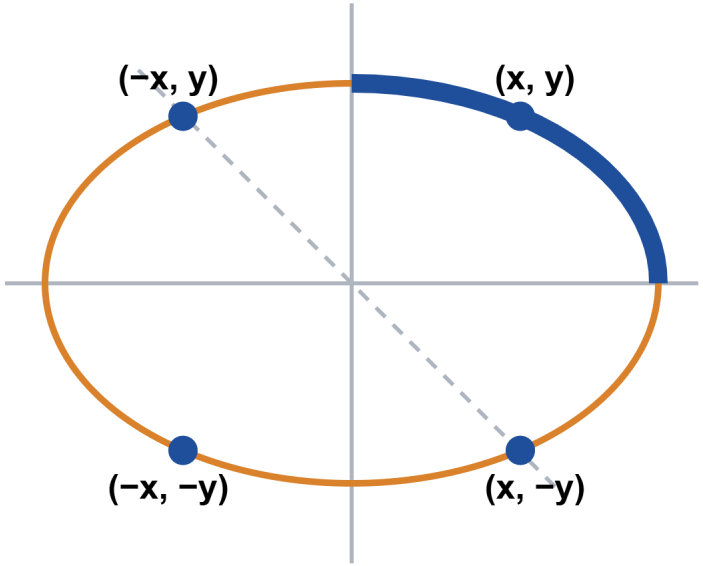
Point is on the ellipse

$F > 0$

Point is outside the ellipse

Same test as the circle: put the midpoint into F and look at the sign.

4-way symmetry (not 8)



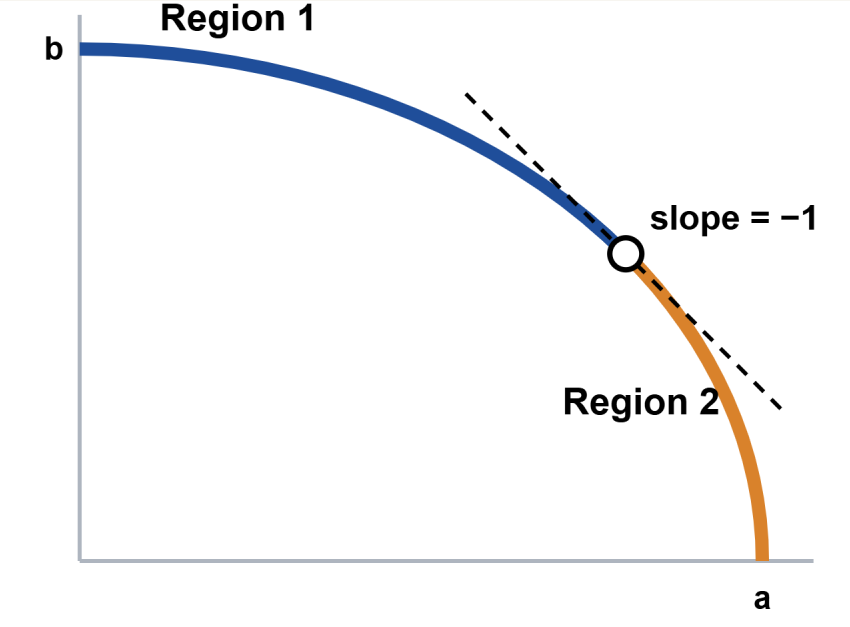
An ellipse is symmetric about the x-axis and the y-axis only.

(±x, ±y): one point gives 4 pixels

It is not symmetric about y = x (the dashed line), because a ≠ b.
Swapping x and y would give a different ellipse.

So we compute a whole quadrant (blue): from (0, b) to (a, 0).

Two regions in the quadrant



Region 1 (blue): |slope| < 1

Flat part. Step x by 1 every time; y stays or goes down 1.

Choices: E or SE.

Region 2 (orange): |slope| > 1

Steep part. Step y down by 1 every time; x stays or goes up 1.

Choices: S or SE.

Why two? The same reason as steep lines: always step along the longer direction, or pixels leave gaps.

Every variable and its meaning

Symbol	Meaning
a, b	Radius along x and along y (integers)
(xc, yc)	Centre of the ellipse
(x_k, y_k)	Current pixel, measured from the centre
F(x, y)	Ellipse function: $b^2x^2 + a^2y^2 - a^2b^2$
$p1_k$	Region 1 decision: F at the midpoint $(x_k + 1, y_k - ½)$
$p2_k$	Region 2 decision: F at the midpoint $(x_k + ½, y_k - 1)$
$2b^2x$, $2a^2y$	Running values: compare them to know when region 1 ends

Proof 1: where region 1 ends

Differentiate $b^2x^2 + a^2y^2 = a^2b^2$ with respect to x:

$2b^2x + 2a^2y \cdot (dy/dx) = 0$

$dy/dx = -2b^2x \div 2a^2y$

$|\text{slope}| < 1$ while $2b^2x < 2a^2y$

Stay in region 1 while $2b^2x < 2a^2y$

Why this is cheap

$2b^2x$ goes up by $2b^2$ each time x goes up.

$2a^2y$ goes down by $2a^2$ each time y goes down.

So the test needs only additions.

Proof 2: the region 1 decision

Choices: $E = (x_k + 1, y_k)$ and $SE = (x_k + 1, y_k - 1)$

Midpoint between them: $(x_k + 1, y_k - \frac{1}{2})$

$p1_k = F(x_k + 1, y_k - \frac{1}{2})$

$= b^2(x_k + 1)^2 + a^2(y_k - \frac{1}{2})^2 - a^2b^2$ ... (1)

$p1_k < 0 \rightarrow E$; $p1_k \geq 0 \rightarrow SE$

Meaning

$p1_k < 0$: the midpoint is inside, so the ellipse passes above it. E is closer.

$p1_k \geq 0$: the midpoint is outside or on it. SE is closer.

Proof 3: region 1 update

Next: $p1_{k+1} = b^2(x_{k+1} + 1)^2 + a^2(y_{k+1} - \tfrac{1}{2})^2 - a^2b^2$, with $x_{k+1} = x_k + 1$

Subtract (1): $p1_{k+1} = p1_k + 2b^2x_{k+1} + b^2 + a^2[(y_{k+1} - \tfrac{1}{2})^2 - (y_k - \tfrac{1}{2})^2]$

E ($y_{k+1} = y_k$): the bracket is 0

SE ($y_{k+1} = y_k - 1$): the bracket is $-2y_{k+1}$

E: $p1_{k+1} = p1_k + 2b^2x_{k+1} + b^2$

SE: $p1_{k+1} = p1_k + 2b^2x_{k+1} - 2a^2y_{k+1} + b^2$

Check the x-part

$b^2[(x_k + 2)^2 - (x_k + 1)^2]$

$= b^2(2x_k + 3)$

$= 2b^2(x_k + 1) + b^2 = 2b^2x_{k+1} + b^2$

Proof 4: the starting value $p1_0$

First pixel: $(0, b)$. First midpoint: $(1, b - \tfrac{1}{2})$

$p1_0 = b^2(1)^2 + a^2(b - \tfrac{1}{2})^2 - a^2b^2$

$= b^2 + a^2(b^2 - b + \tfrac{1}{4}) - a^2b^2$

$= b^2 + a^2b^2 - a^2b + \tfrac{1}{4}a^2 - a^2b^2$

$$p1_0 = b^2 - a^2b + \tfrac{1}{4}a^2$$

Example

$a = 8, b = 6$:

$p1_0 = 36 - 64(6) + \tfrac{1}{4}(64)$

$= 36 - 384 + 16 = -332$

Proof 5: region 2 decision and update

Choices: $S = (x_k, y_k - 1)$ and $SE = (x_k + 1, y_k - 1)$

Midpoint: $(x_k + \frac{1}{2}, y_k - 1)$; $p2_k = F(x_k + \frac{1}{2}, y_k - 1)$

$p2_k > 0 \rightarrow$ midpoint outside $\rightarrow S$; $p2_k \leq 0 \rightarrow SE$

Same subtraction as before, now y always goes down by 1:

S: $p2_{k+1} = p2_k - 2a^2y_{k+1} + a^2$

SE: $p2_{k+1} = p2_k + 2b^2x_{k+1} - 2a^2y_{k+1} + a^2$

Check the y-part

$a^2[(y_k - 2)^2 - (y_k - 1)^2]$

$= a^2(-2y_k + 3)$

$= -2a^2y_{k+1} + a^2$

x-part for SE: $b^2[(x_k + 3/2)^2 - (x_k + \frac{1}{2})^2] = 2b^2x_{k+1}$

Proof 6: the starting value $p2_0$

Region 2 starts at the last region 1 pixel (x_0, y_0).

Its first midpoint is $(x_0 + ½, y_0 − 1)$:

$$p2_0 = b^2(x_0 + ½)^2 + a^2(y_0 − 1)^2 − a^2b^2$$

Compute it once, directly from F, then keep updating.

Example

$a = 8$, $b = 6$, last region 1 pixel $(7, 3)$:

$p2_0 = 36(7.5)^2 + 64(2)^2 − 2304$

$= 2025 + 256 − 2304 = −23$

Formula card: midpoint ellipse

Region 1: start at (0, b)

$$p1_0 = b^2 - a^2 b + \tfrac{1}{4}a^2$$

p1 < 0 → E:

$$p1 + 2b^2 x_{k+1} + b^2$$

p1 ≥ 0 → SE:

$$p1 + 2b^2 x_{k+1} - 2a^2 y_{k+1} + b^2$$

Stay while $2b^2 x < 2a^2 y$

Region 2: start at last pixel (x_0, y_0)

$$p2_0 = b^2(x_0 + \tfrac{1}{2})^2 + a^2(y_0 - 1)^2 - a^2 b^2$$

p2 > 0 → S:

$$p2 - 2a^2 y_{k+1} + a^2$$

p2 ≤ 0 → SE:

$$p2 + 2b^2 x_{k+1} - 2a^2 y_{k+1} + a^2$$

Stop when y = 0

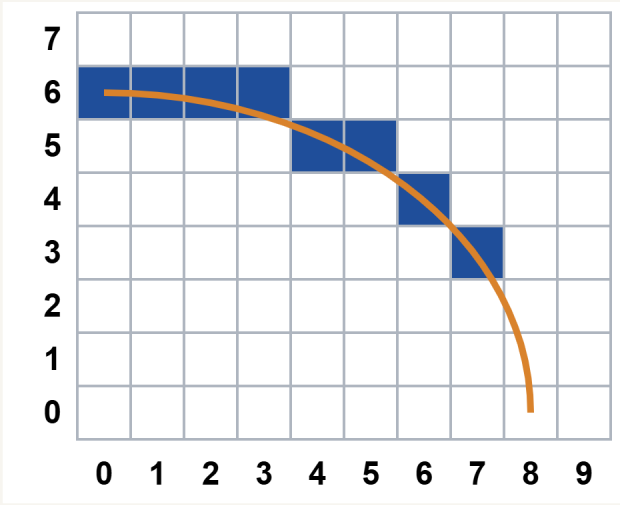
Midpoint ellipse algorithm

1. Input a, b and centre (xc, yc). Start at x = 0, y = b.
2. p1 = $b^2 - a^2b + \frac{1}{4}a^2$; px = 0; py = $2a^2y$. Plot the 4 symmetric points.
3. Region 1, while px < py: x = x + 1, px = px + $2b^2$. If p1 < 0: p1 = p1 + px + b^2. Else: y = y − 1, py = py − $2a^2$, p1 = p1 + px − py + b^2. Plot 4 points.
4. p2 = $b^2(x + \frac{1}{2})^2 + a^2(y - 1)^2 - a^2b^2$
5. Region 2, while y > 0: y = y − 1, py = py − $2a^2$. If p2 > 0: p2 = p2 − py + a^2. Else: x = x + 1, px = px + $2b^2$, p2 = p2 + px − py + a^2. Plot 4 points.
6. Every plotted point is shifted by (xc, yc).

Example: a = 8, b = 6 (region 1)

Start (0, 6). $p1_0 = -332$. $2b^2 = 72$, $2a^2 = 128$. Start: $2a^2y = 768$.

k	$p1_k$	Decision	(x_{k+1}, y_{k+1})	$2b^2x_{k+1}$	$2a^2y_{k+1}$
0	-332	p1 < 0 → E	(1, 6)	72	768
1	-224	p1 < 0 → E	(2, 6)	144	768
2	-44	p1 < 0 → E	(3, 6)	216	768
3	208	p1 ≥ 0 → SE	(4, 5)	288	640
4	-108	p1 < 0 → E	(5, 5)	360	640
5	288	p1 ≥ 0 → SE	(6, 4)	432	512
6	244	p1 ≥ 0 → SE	(7, 3)	504	384

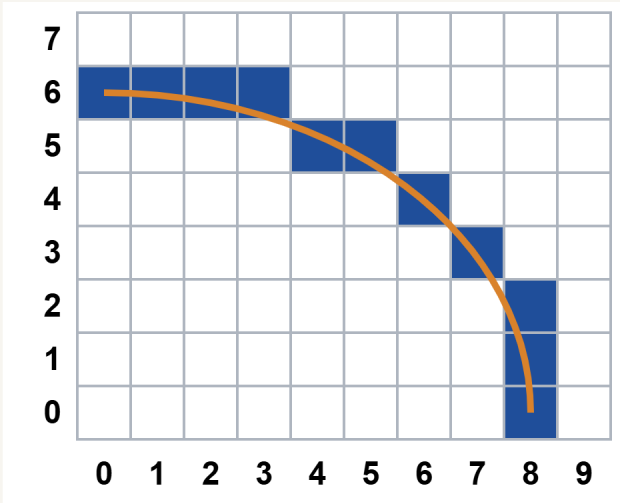


Example: a = 8, b = 6 (region 2)

Start at (7, 3). $p2_0 = 36(7.5)^2 + 64(2)^2 - 2304 = -23$.

k	$p2_k$	Decision	(x_{k+1}, y_{k+1})	$2b^2x_{k+1}$	$2a^2y_{k+1}$
0	-23	p2 ≤ 0 → SE	(8, 2)	576	256
1	361	p2 > 0 → S	(8, 1)	576	128
2	297	p2 > 0 → S	(8, 0)	576	0

Quadrant pixels: (0,6) (1,6) (2,6) (3,6) (4,5) (5,5) (6,4) (7,3) (8,2) (8,1) (8,0)



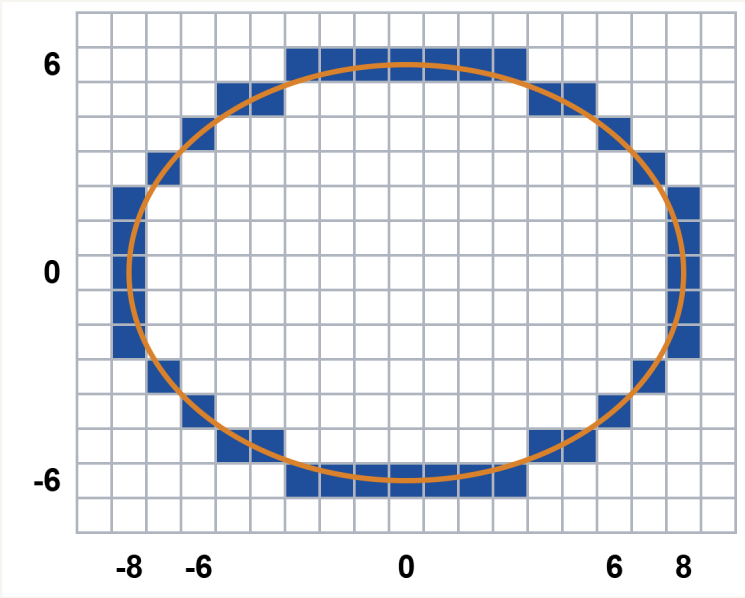
Using symmetry: the whole ellipse

Each quadrant pixel gives 4 pixels. From (4, 5):

(4, 5) (−4, 5) (4, −5) (−4, −5)

For centre (xc, yc) = (20, 15): (4, 5) → (24, 20).

11 quadrant pixels gave the whole ellipse: 40 different pixels.

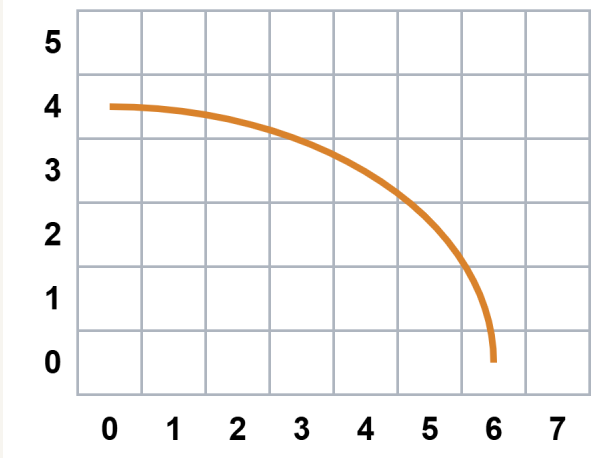


Class exercise: your turn

Use the midpoint ellipse algorithm for a = 6, b = 4, centre (0, 0).

1. Find $p1_0$, $2b^2$ and $2a^2$
2. Region 1 table until $2b^2x \geq 2a^2y$
3. Find $p2_0$ and the region 2 table
4. Shade the quadrant pixels

Time: 8 minutes.



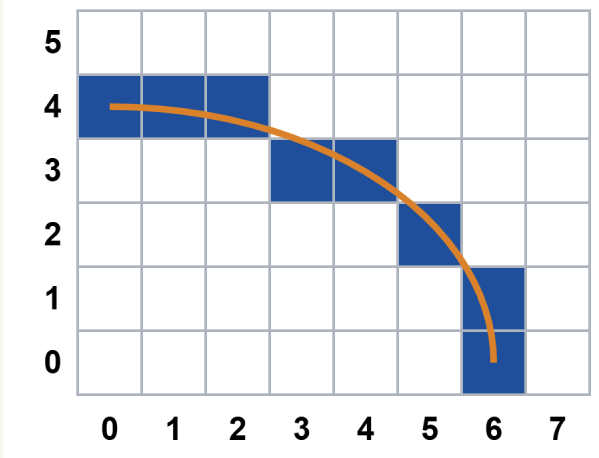
Exercise answer: a = 6, b = 4

$p1_0 = 16 - 36(4) + 9 = -119$. $2b^2 = 32$, $2a^2 = 72$.

k	$p1_k$	Decision	(x_{k+1}, y_{k+1})	$2b^2x_{k+1}$	$2a^2y_{k+1}$
0	-119	$p1 < 0 \rightarrow$ E	(1, 4)	32	288
1	-71	$p1 < 0 \rightarrow$ E	(2, 4)	64	288
2	9	$p1 \geq 0 \rightarrow$ SE	(3, 3)	96	216
3	-95	$p1 < 0 \rightarrow$ E	(4, 3)	128	216
4	49	$p1 \geq 0 \rightarrow$ SE	(5, 2)	160	144

$p2_0 = 16(5.5)^2 + 36(1)^2 - 576 = -56$

k	$p2_k$	Decision	(x_{k+1}, y_{k+1})	$2b^2x_{k+1}$	$2a^2y_{k+1}$
0	-56	$p2 \leq 0 \rightarrow$ SE	(6, 1)	192	72
1	100	$p2 > 0 \rightarrow$ S	(6, 0)	192	0



Midpoint ellipse in C++ with OpenGL

```
void midpointEllipse(int xc, int yc, int a, int b) {
    long a2 = a * a, b2 = b * b;
    int x = 0, y = b;  long px = 0, py = 2 * a2 * y;
    long p = lround(b2 - a2 * b + 0.25 * a2);            // p1
    plot4(xc, yc, x, y);
    while (px < py) {                                        // region 1
        x++;  px += 2 * b2;
        if (p < 0) p += b2 + px;
        else { y--;  py -= 2 * a2;  p += b2 + px - py; }
        plot4(xc, yc, x, y);
    }
    p = lround(b2 * (x + 0.5) * (x + 0.5) + a2 * (y - 1) * (y - 1) - a2 * b2);
    while (y > 0) {                                          // region 2
        y--;  py -= 2 * a2;
        if (p > 0) p += a2 - py;
        else { x++;  px += 2 * b2;  p += a2 - py + px; }
        plot4(xc, yc, x, y);
    }
}
```

Part 2

Filling a shape

Boundary fill and flood fill

Two ways to fill a region

Seed fill (today)

Start at one pixel inside (the seed) and spread to its neighbours until the region is full.

- **Boundary fill**
- **Flood fill**

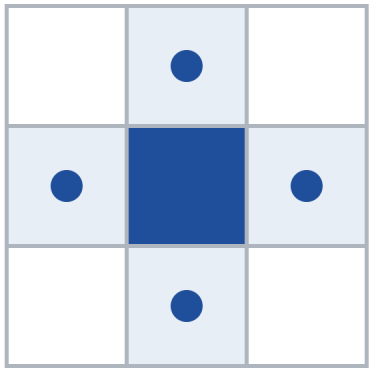
Used in paint programs: the bucket tool.

Scan-line fill (next class)

Go row by row; find where each row crosses the polygon edges and fill between the pairs.

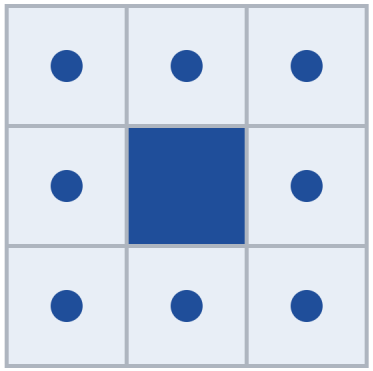
Used for polygons whose vertices we know.

Neighbours: 4-connected and 8-connected



4-connected

Right, left, up, down: (x±1, y) and (x, y±1).



8-connected

Also the 4 diagonals: (x±1, y±1).

Boundary fill

The region is closed by a boundary of one colour. Paint every pixel inside it.

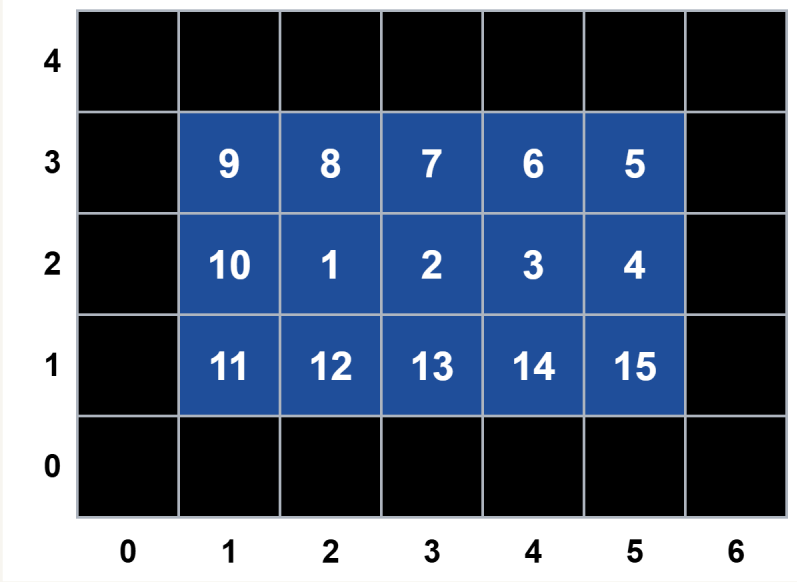
```
boundaryFill4(x, y, fill, boundary):
    c = getPixel(x, y)
    if c ≠ boundary and c ≠ fill:
        setPixel(x, y, fill)
        boundaryFill4(x + 1, y, fill, boundary)
        boundaryFill4(x - 1, y, fill, boundary)
        boundaryFill4(x, y + 1, fill, boundary)
        boundaryFill4(x, y - 1, fill, boundary)
```

Two stop rules

Stop at the boundary colour.

Stop at a pixel already painted with the fill colour, or we would loop for ever.

Boundary fill: the order pixels are painted



Black = boundary. Seed = (2, 2). Neighbour order: right, left, up, down.

The fill runs right as far as it can (1 → 4), then each call that returns tries its next direction.

All 15 inside pixels painted; the boundary is never crossed.

Flood fill

The region is all connected pixels of one old colour, whatever colour the border is. Replace that colour.

```
floodFill4(x, y, fill, old):
    if getPixel(x, y) == old:
        setPixel(x, y, fill)
        floodFill4(x + 1, y, fill, old)
        floodFill4(x - 1, y, fill, old)
        floodFill4(x, y + 1, fill, old)
        floodFill4(x, y - 1, fill, old)
```

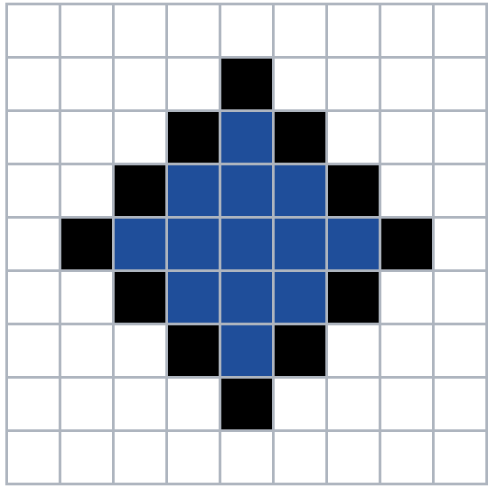
When to use it

The border has many colours (a map, a cartoon), so there is no single boundary colour to stop at.

Stop rule: the pixel is not the old colour.

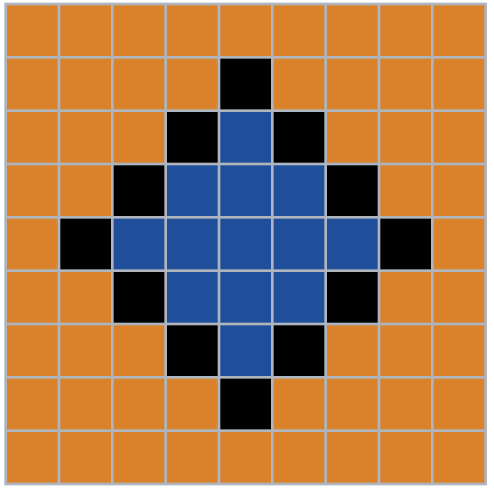
fill must differ from old, or it never stops.

Watch out: 8-connected fill can leak



4-connected

Stays inside: the diagonal steps of the border are enough to stop it.



8-connected

Slips between two border pixels that touch only at a corner and paints outside (orange).

Boundary fill and flood fill in C++

```
void boundaryFill4(int x, int y, Color fill, Color boundary) {
    Color c = getPixel(x, y);
    if (c ≠ boundary && c ≠ fill) {
        setPixel(x, y, fill);
        boundaryFill4(x + 1, y, fill, boundary);  boundaryFill4(x - 1, y, fill, boundary);
        boundaryFill4(x, y + 1, fill, boundary);  boundaryFill4(x, y - 1, fill, boundary);
    }
}
void floodFill4(int x, int y, Color fill, Color old) {
    if (getPixel(x, y) == old) {
        setPixel(x, y, fill);
        floodFill4(x + 1, y, fill, old);  floodFill4(x - 1, y, fill, old);
        floodFill4(x, y + 1, fill, old);  floodFill4(x, y - 1, fill, old);
    }
}
```

Problems with recursive seed fill

Deep recursion

One call per pixel. A 500 × 500 region can need 250,000 calls in a chain: the stack overflows (Python stops at about 1,000).

Repeated checks

Every pixel is tested up to 4 (or 8) times by its neighbours.

Fixes

- Use your own stack or queue instead of recursion
- Span fill: fill a whole row run at once, push only the rows above and below

Boundary fill vs flood fill

	Boundary fill	Flood fill
Region defined by	A border of one colour	Connected pixels of one old colour
Stops at	Boundary colour or fill colour	Any pixel not the old colour
Inputs	Seed, fill colour, boundary colour	Seed, fill colour, old colour
Border with many colours	Fails (leaks past other colours)	Works
Typical use	Filling a shape we just drew	Paint-bucket tool, maps, cartoons
Both	4- or 8-connected; recursive or with a stack	4- or 8-connected; recursive or with a stack

Key points from today

1. $F(x, y) = b^2x^2 + a^2y^2 - a^2b^2$: negative inside, zero on, positive outside.
2. An ellipse has 4-way symmetry, so we compute one quadrant in two regions.
3. Region 1 (step x) while $2b^2x < 2a^2y$; region 2 (step y) until $y = 0$.
4. $p1_0 = b^2 - a^2b + \frac{1}{4}a^2$; $p2_0 = b^2(x + \frac{1}{2})^2 + a^2(y - 1)^2 - a^2b^2$.
5. Seed fill spreads from one inside pixel to its 4 or 8 neighbours.
6. Boundary fill stops at one border colour; flood fill replaces one old colour.
7. 8-connected fill can leak through diagonal gaps; recursion can overflow the stack.

Practice questions

1. Use the midpoint ellipse algorithm for a = 10, b = 6. Show both region tables.
2. Use the midpoint ellipse algorithm for a = 4, b = 2, centre (10, 10).
3. Derive $p1_0 = b^2 - a^2b + \frac{1}{4}a^2$.
4. Why does an ellipse need two regions but a circle does not?
5. Write the 4-connected boundary fill algorithm. How is flood fill different?
6. Draw a border where an 8-connected fill leaks but a 4-connected fill does not.

Coming up

Next lecture

Scan-line polygon fill

Edge intersections, the edge table and active edge list, vertex special cases, a worked example

Lab 3

Midpoint circle and ellipse

Code both in Python with PyOpenGL on the pixel-grid viewer; try different a and b

Questions?

Thank you

Ekramul Haque Amin · Lecturer, AI and Data Science